

Tipos Algébricos

Programação Funcional



Tipos Algébricos

- Permitem a definição de tipos mais complexos
 - Aumenta a expressividade da linguagem
- Tipos já estudados
 - Básicos (primitivos): Int, Float, Double, Bool e Char
 - Compostos: listas, tuplas e funções $t1 \rightarrow t2$
- Representação
 - Enumerações em algumas linguagens
 - Tipos compostos por outros tipos (unions)
 - Estruturas de dados complexas (árvore, grafo, ...)



Tipos Algébricos

- Sintaxe de definição
 - `data <nome_tipo> = <val1> | <val2> | ... | <valN>`
 - `<val>` é o construtor de valor
 - Define os valores do tipo
 - Pode ter componentes de valor
- Exemplo

```
data Dia_Util = Segunda  
              | Terca  
              | Quarta  
              | Quinta  
              | Sexta  
              deriving (Eq,Show)
```



Tipos Algébricos

```
data Publicacao = Livro Int String  
                | Revista String Int  
                | Artigo String [String]  
                deriving (Show)
```

```
a = Revista "Wired" 137
```

```
b = Artigo "Haskell na Web" ["Ana Silva", "Rui Santos"]
```



Tipos Algébricos

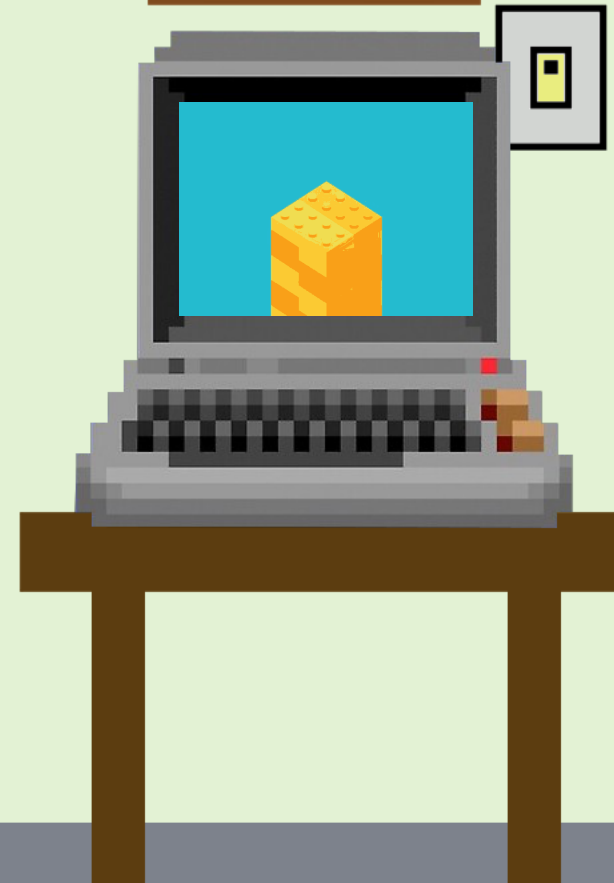
■ Sintaxe de registro

```
data Pessoa= Pessoa String String Int deriving (Show)
```

```
idade :: Pessoa -> Int
```

```
idade (Pessoa _ _ i) = i
```

```
data Pessoa= Pessoa { nome :: String  
                      , sobrenome :: String  
                      , idade :: Int  
                      } deriving (Show)
```



Tipos Recursivos

- Podemos utilizar o próprio tipo definido como um componente de valor de um tipo algébrico

```
data Expr = Lit Int  
          | Add Expr Expr  
          | Sub Expr Expr
```

```
eval :: Expr -> Int  
eval (Lit n) = n  
eval (Add e1 e2) = (eval e1) + (eval e2)  
eval (Sub e1 e2) = (eval e1) - (eval e2)
```

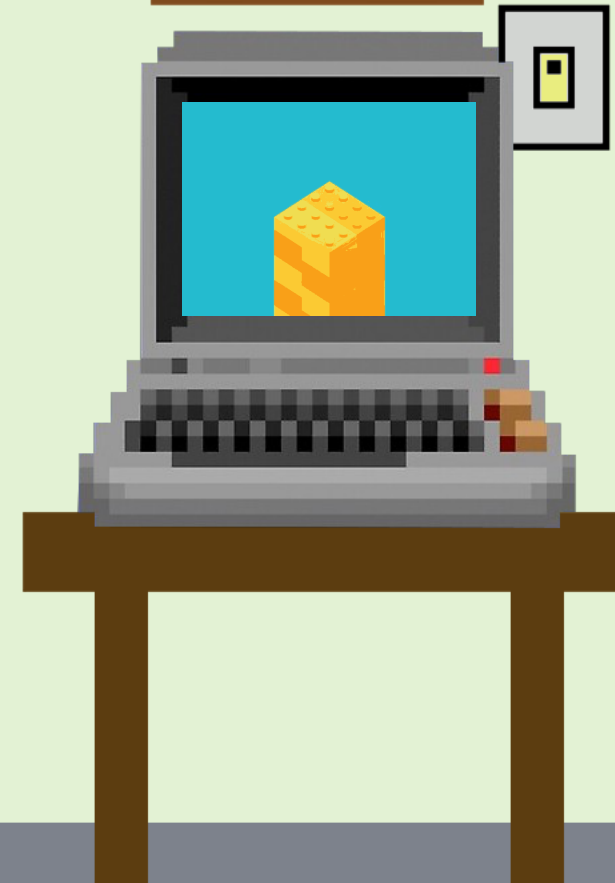


Tipos Algébricos Polimórficos

- As definições de tipos algébricos podem conter tipos variáveis como t , u , etc.
 - `data Pares t = Par t t`
- Exemplo com lista
 - Lista vazia
 - Lista não vazia

```
data Lista a = Nil | Cons a (Lista a)
```

```
comprimento :: Lista a -> Int  
comprimento Nil = 0  
comprimento (Cons head tail) = 1 + comprimento tail
```



Exercícios

- Defina o tipo Retângulo com base no tipo Figura
 - Programe uma função que calcule o perímetro e área das figuras geométricas definidas

```
type Ponto = (Float, Float)
data Figura = Circulo Ponto Float
            | Quadrado Ponto Float
```

- Defina o tipo Sólido, capaz de representar os elementos Esfera, Cubo e Pirâmide
 - Crie uma função que calcule o volume dos sólidos



Exercícios

- Criar uma agenda eletrônica com os registros
 - Contato (nome, telefone e endereço)
 - Compromisso (data, título e descrição)
- Podem ser realizadas as seguintes operações
 - Inserir registro (não deve aceitar dois compromissos na mesma data)
 - Remover registro
 - Consultar registro

